

Computer Networks Lab 3

Muhammad Hassan Khalid
21L-5692

Taking Wireshark for a Test Run

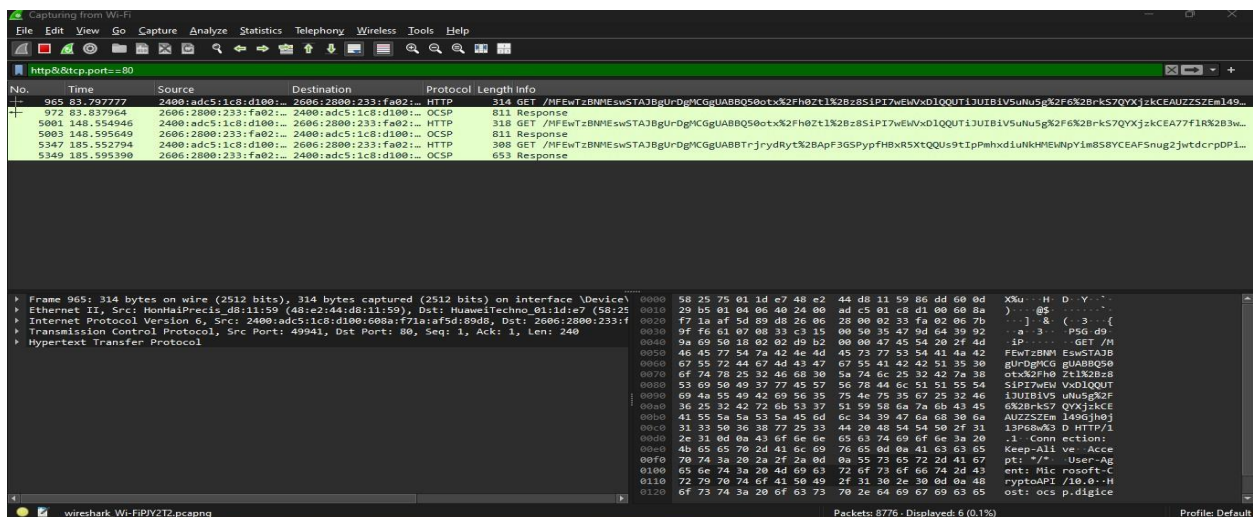
Not Found

The requested URL /favicon.ico was not found on this server.

Congratulations! You've downloaded the first Wireshark lab file!

Wireshark Filters

- `http && tcp.port==80`



- tcp || ip.addr==192.168.1.2

Wireshark packet capture showing a list of TCP packets. The filter is `tcp || ip.addr==192.168.1.2`. The list shows packets from 192.168.1.2 to 192.168.1.1. The detailed view shows packet 965, which is a TCP PDU reassembled in 9164. The raw data is displayed in hexadecimal and ASCII.

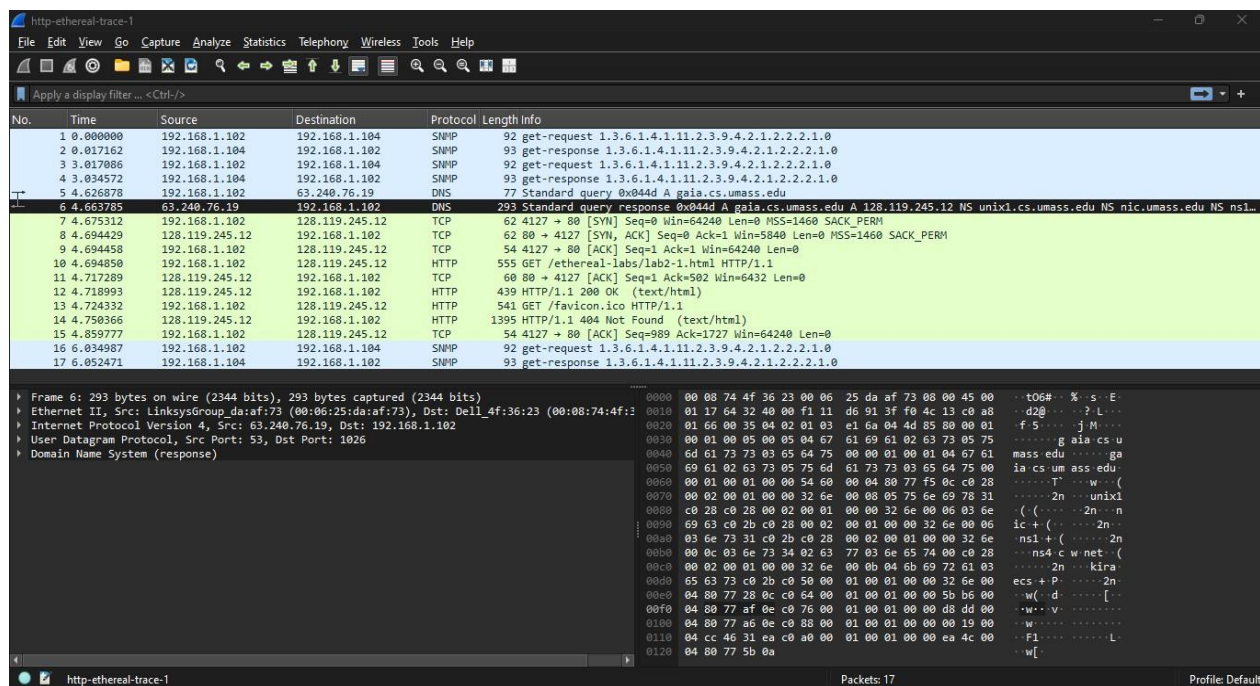
- !(ip.src==192.168.1.2)

Wireshark packet capture showing a list of TCP packets. The filter is `!(ip.src==192.168.1.2)`. The list shows packets from 192.168.1.2 to 192.168.1.1. The detailed view shows packet 965, which is a TCP PDU reassembled in 9164. The raw data is displayed in hexadecimal and ASCII.

In-Lab Statement 1: Analyzing HTTP Protocol

1- Protocols appearing in the unfiltered packet-listing window:

- SNMP: Simple Network Management Protocol
- DNS: Domain Name System
- TCP (Transmission Control Protocol)
- HTTP (Hypertext Transfer Protocol)



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.102	192.168.1.104	SNMP	92	get-request 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0
2	0.017162	192.168.1.104	192.168.1.102	SNMP	93	get-response 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0
3	0.017086	192.168.1.102	192.168.1.104	SNMP	92	get-request 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0
4	0.034572	192.168.1.104	192.168.1.102	SNMP	93	get-response 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0
5	4.626878	192.168.1.102	63.240.76.19	DNS	77	Standard query 0x044d A gaia.cs.umass.edu
6	4.663785	63.240.76.19	192.168.1.102	DNS	293	Standard query response 0x044d A gaia.cs.umass.edu A 128.119.245.12 NS unix1.cs.umass.edu NS nic.umass.edu NS ns1.
7	4.675312	192.168.1.102	128.119.245.12	TCP	62	4127 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM
8	4.694429	128.119.245.12	192.168.1.102	TCP	62	80 → 4127 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM
9	4.694458	192.168.1.102	128.119.245.12	TCP	54	4127 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
10	4.694850	192.168.1.102	128.119.245.12	HTTP	555	GET /ethereal-labs/lab2-1.html HTTP/1.1
11	4.717289	128.119.245.12	192.168.1.102	TCP	60	80 → 4127 [ACK] Seq=1 Ack=502 Win=6432 Len=0
12	4.718993	128.119.245.12	192.168.1.102	HTTP	439	HTTP/1.1 200 OK (text/html)
13	4.724332	192.168.1.102	128.119.245.12	HTTP	541	GET /favicon.ico HTTP/1.1
14	4.750366	128.119.245.12	192.168.1.102	HTTP	1395	HTTP/1.1 404 Not Found (text/html)
15	4.859777	192.168.1.102	128.119.245.12	TCP	54	4127 → 80 [ACK] Seq=989 Ack=1727 Win=64240 Len=0
16	6.034987	192.168.1.102	192.168.1.104	SNMP	92	get-request 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0
17	6.052471	192.168.1.104	192.168.1.102	SNMP	93	get-response 1.3.6.1.4.1.11.2.3.9.4.2.1.2.2.2.1.0

2- Time taken from HTTP GET to HTTP OK reply:

From the packet trace provided, the GET request (Frame 13) was sent at 4.724332000 seconds, and the HTTP response (Frame 14) was received at 4.750366000 seconds. Therefore, the round-trip time is approximately:

$$4.750366 - 4.724332 = 0.026034 \text{ seconds}$$

$$0.026034 * 1000 = 26.034 \text{ ms}$$

The image displays two screenshots of the Wireshark network protocol analyzer. The top screenshot shows the packet list with four HTTP requests. The bottom screenshot shows the details of Frame 13, which is the response to the third request (GET /favicon.ico), resulting in a 404 Not Found error.

Packet List (Top Screenshot):

No.	Time	Source	Destination	Protocol	Length	Info
10	4.694850	192.168.1.102	128.119.245.12	HTTP	555	GET /ethtereal-labs/lab2-1.html HTTP/1.1
12	4.718993	128.119.245.12	192.168.1.102	HTTP	439	HTTP/1.1 200 OK (text/html)
13	4.724332	192.168.1.102	128.119.245.12	HTTP	541	GET /favicon.ico HTTP/1.1
14	4.750366	128.119.245.12	192.168.1.102	HTTP	1395	HTTP/1.1 404 Not Found (text/html)

Frame 13 Details (Bottom Screenshot):

Frame 13: 541 bytes on wire (4328 bits), 541 bytes captured (4328 bits) on interface 0

Encapsulation type: Ethernet (1)

Arrival Time: Sep 23, 2003 10:29:47.867870000 Pakistan Standard Time

UTC Arrival Time: Sep 23, 2003 05:29:47.867870000 UTC

Epoch Arrival Time: 1064294987.867870000

[Time shift for this packet: 0.000000000 seconds]

[Time delta from previous captured frame: 0.005339000 seconds]

[Time delta from previous displayed frame: 0.005339000 seconds]

[Time since reference or first frame: 4.724332000 seconds]

Frame Number: 13

Frame Length: 541 bytes (4328 bits)

Capture Length: 541 bytes (4328 bits)

[Frame is marked: False]

[Frame is ignored: False]

[Protocols in frame: eth:ethertype:ip:tcp:http]

[Coloring Rule Name: HTTP]

HTTP Hypertext Transfer Protocol

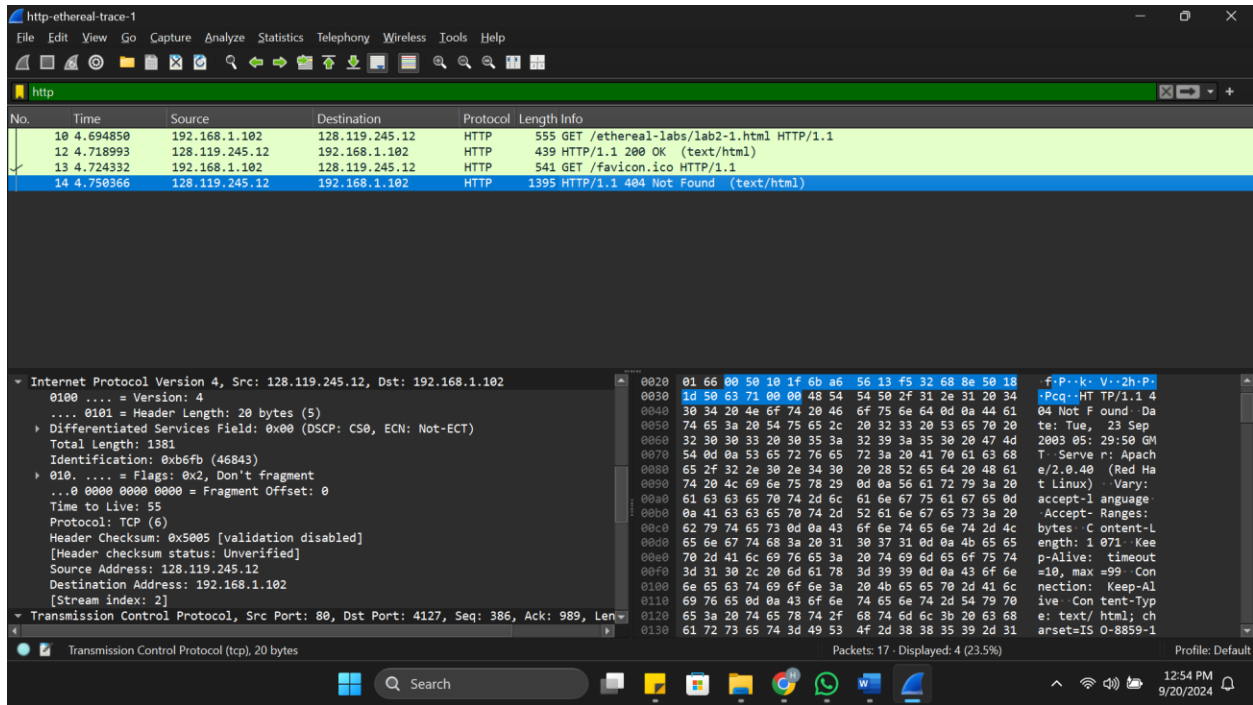
GET /favicon.ico HTTP/1.1

Host: g.aisa.cs.u

User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.0.2) Gecko/20021120 Netscape/7.0.1. Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,image/jpeg;q=0.8

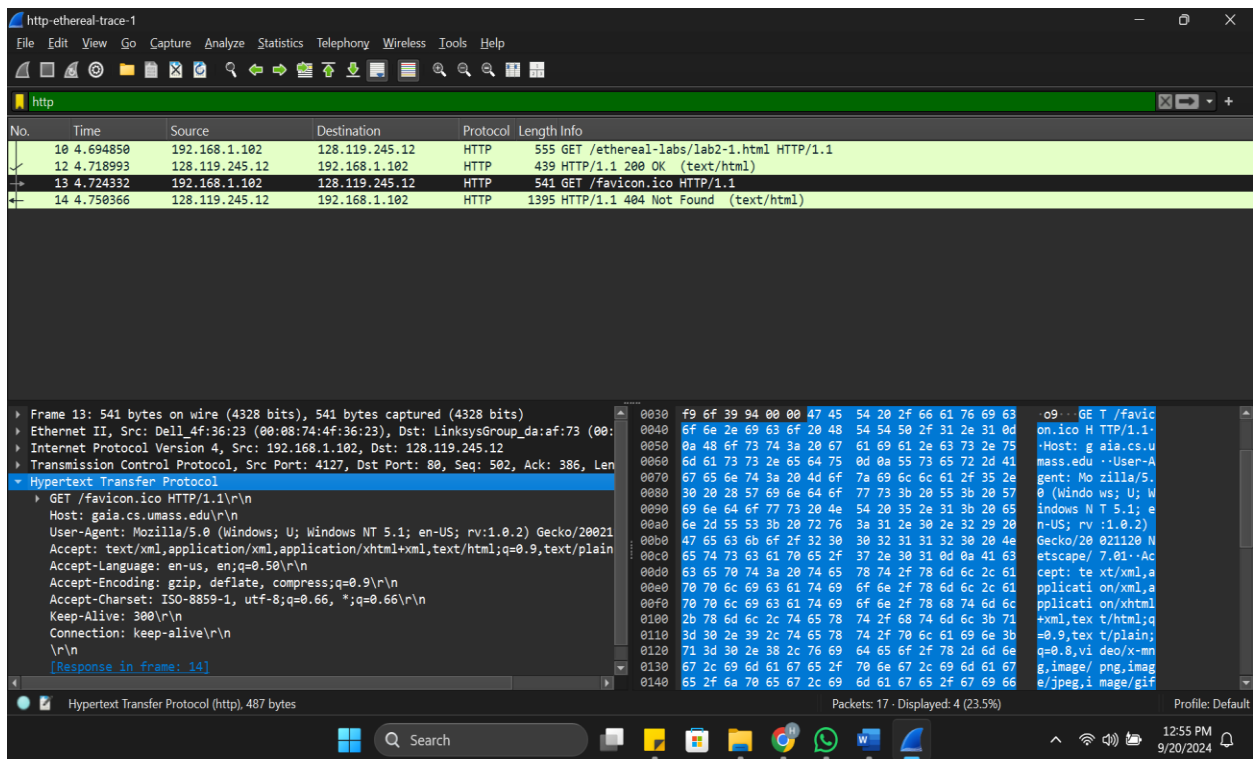
3- Was the second GET request successful?

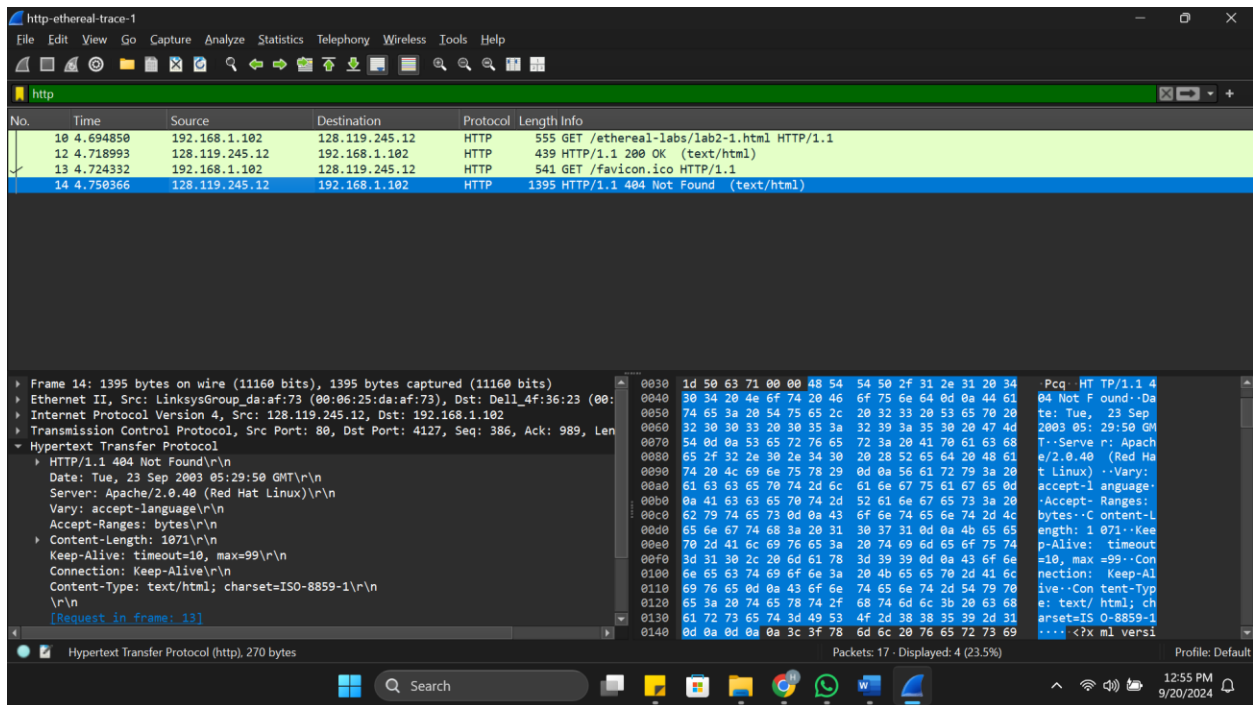
No, the second GET request was not successful. The response to the GET request for /favicon.ico resulted in an HTTP 404 "Not Found" error, as indicated in Frame 14.



4- HTTP version used by browser and server:

- **Browser:** The request in Frame 13 shows HTTP/1.1.
- **Server:** The response in Frame 14 also shows HTTP/1.1.





5- Languages the browser can accept: From the GET request in Frame 13:

Accept-Language: en-us, en;q=0.50

```
Accept-Language: en-us, en;q=0.50\r\n
```

6- IP addresses:

- **Computer's IP:** 192.168.1.102
- **gaia.cs.umass.edu server's IP:** 128.119.245.12

```
[Header checksum status: Unverified]
Source Address: 192.168.1.102
Destination Address: 128.119.245.12
[Stream index: 2]
Transmission Control Protocol, Src Port: 4127, Dst Port: 80, Seq: 1, Ack: 1, Len: 501
```

7- MAC addresses:

- **Computer's MAC:** 00:08:74:4f:36:23 (Dell_4f:36:23)
- **Server's MAC:** 00:06:25:da:af:73 (LinksysGroup_da:af:73)

```

[Coloring Rule String: http || tcp.port == 80 || http2]
▼ Ethernet II, Src: LinksysGroup_da:af:73 (00:06:25:da:af:73), Dst: Dell_4f:36:23 (00:08:74:4f:36:23)
  ▸ Destination: Dell_4f:36:23 (00:08:74:4f:36:23)
  ▸ Source: LinksysGroup_da:af:73 (00:06:25:da:af:73)
    Type: IPv4 (0x0800)
    [Stream index: 1]

```

8- Sending and receiving port numbers:

- **Source Port:** 4127 (Computer)
- **Destination Port:** 80 (the HTTP server)
- Port 80 represents the default port used for HTTP web traffic.

```

[Stream index: 2]
Transmission Control Protocol, Src Port: 4127, Dst Port: 80, Seq: 1, Ack: 1, Len: 501
Source Port: 4127
Destination Port: 80
[Stream index: 0]

```

9- Status code returned from the server:

The server returned an HTTP 404 status code, meaning the requested resource /favicon.ico was not found.

12	4.718993	128.119.245.12	192.168.1.102	HTTP	439	HTTP/1.1 200 OK (text/html)
13	4.724332	192.168.1.102	128.119.245.12	HTTP	541	GET /favicon.ico HTTP/1.1
14	4.750366	128.119.245.12	192.168.1.102	HTTP	1395	HTTP/1.1 404 Not Found (text/html)

10- Last modified date of the HTML file:

The response does not indicate when the HTML file was last modified, as it is not part of the HTTP 404 error response.

11- Total bytes returned to the browser:

The content length in the response header is 1071 bytes, which refers to the body of the response (the HTML error page).

In-Lab Statement 2 : Analyzing HTTP Protocol

The HTTP CONDITIONAL GET/response interaction

1. **Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an “IF-MODIFIED-SINCE” line in the HTTP GET?**

In the first HTTP GET request (Frame 8), there is **no "IF-MODIFIED-SINCE"** line. This is a standard GET request from the browser to fetch the resource /ethereal-labs/lab2-2.html from the server.

2. **Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell from the Packet Bytes Window?**

Yes, the server did **explicitly return the contents of the file**. We can tell because the server response (Frame 10) has a status code of **200 OK**, indicating that the requested resource was successfully delivered. Additionally, in the Packet Bytes window, we can see the file contents, including HTML tags like <html> and the message "Congratulations again! Now you've downloaded the file lab2-2.html", confirming the content was transferred.

3. **Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an “IF-MODIFIED-SINCE:” line in the HTTP GET? If so, what information follows the “IF-MODIFIED-SINCE:” header? What is meant by this information?**

In the second HTTP GET request (Frame 14), **yes**, there is an "IF-MODIFIED-SINCE" line. The information that follows the header is the timestamp: **Tue, 23 Sep 2003 05:35:00 GMT**.

This information indicates that the browser is telling the server to send the file only if it has been modified since this timestamp. This is part of the **Conditional GET** mechanism used to optimize resource retrieval by avoiding unnecessary data transfer if the file has not changed.

```
Connection: keep-alive\r\n
If-Modified-Since: Tue, 23 Sep 2003 05:35:00 GMT\r\n
If-None-Match: "1bfef-173-8f4ae900"\r\n
Cache-Control: max-age=0\r\n
```


4. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain your answer.

The server responded with the HTTP status code **304 Not Modified**. This means that the server did **not return the contents of the file** because it determined that the file had not been modified since the specified date and time in the "IF-MODIFIED-SINCE" header. This is why the content is not explicitly returned—only the headers are sent to inform the browser that the cached version can be used.

The image shows a Wireshark packet capture of an HTTP transaction. The packet list pane at the top shows five packets. Packet 15 is the response of interest, an HTTP 304 Not Modified status. The packet details pane on the left shows the expanded structure of packet 15, including the Hypertext Transfer Protocol section with the status code and various headers. The packet bytes pane on the right shows the raw data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
8	2.331268	192.168.1.102	128.119.245.12	HTTP	555	GET /ethereal-labs/lab2-2.html HTTP/1.1
10	2.357902	128.119.245.12	192.168.1.102	HTTP	739	HTTP/1.1 200 OK (text/html)
14	5.517390	192.168.1.102	128.119.245.12	HTTP	668	GET /ethereal-labs/lab2-2.html HTTP/1.1
15	5.540216	128.119.245.12	192.168.1.102	HTTP	243	HTTP/1.1 304 Not Modified

Frame 15: 243 bytes on wire (1944 bits), 243 bytes captured (1944 bits) on interface 0
Ethernet II, Src: LinksysGroup da:af:73 (00:06:25:da:af:73), Dst: Dell_4f:36:23 (00:01:00:00:00:00)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.1.102
Transmission Control Protocol, Src Port: 80, Dst Port: 4247, Seq: 686, Ack: 1116, Len: 243
Hypertext Transfer Protocol
 HTTP/1.1 304 Not Modified\r\n Date: Tue, 23 Sep 2003 05:35:53 GMT\r\n Server: Apache/2.0.40 (Red Hat Linux)\r\n Connection: Keep-Alive\r\n Keep-Alive: timeout=10, max=99\r\n ETag: "1bfef-173-8f4ae900"\r\n \r\n [Request in frame: 14]
 [Time since request: 0.022826000 seconds]
 [Request URI: /ethereal-labs/lab2-2.html]
 [Full request URI: http://gaia.cs.umass.edu/ethereal-labs/lab2-2.html]

5. How many HTTP GET request messages did your browser send?

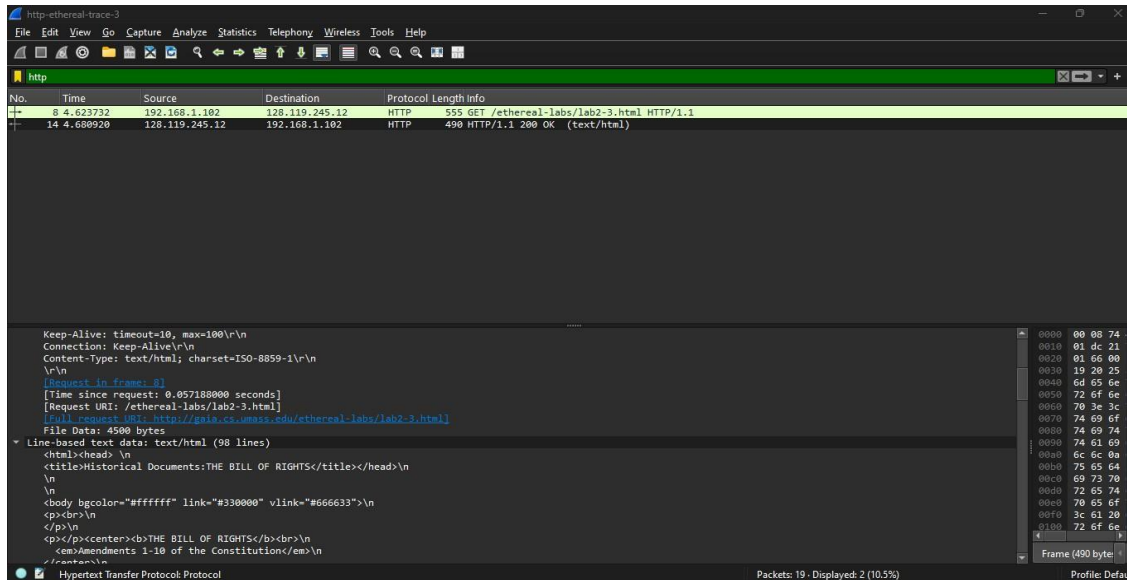
To find the number of HTTP GET request messages, you need to filter for `http.request.method == "GET"`. The filtered trace will show that there is **1 HTTP GET request** message sent by the browser.

The image shows a Wireshark packet capture window with the filter `http.request.method == "GET"` applied. The packet list pane shows only one packet, which is an HTTP GET request for /ethereal-labs/lab2-3.html.

No.	Time	Source	Destination	Protocol	Length	Info
8	4.623732	192.168.1.102	128.119.245.12	HTTP	555	GET /ethereal-labs/lab2-3.html HTTP/1.1

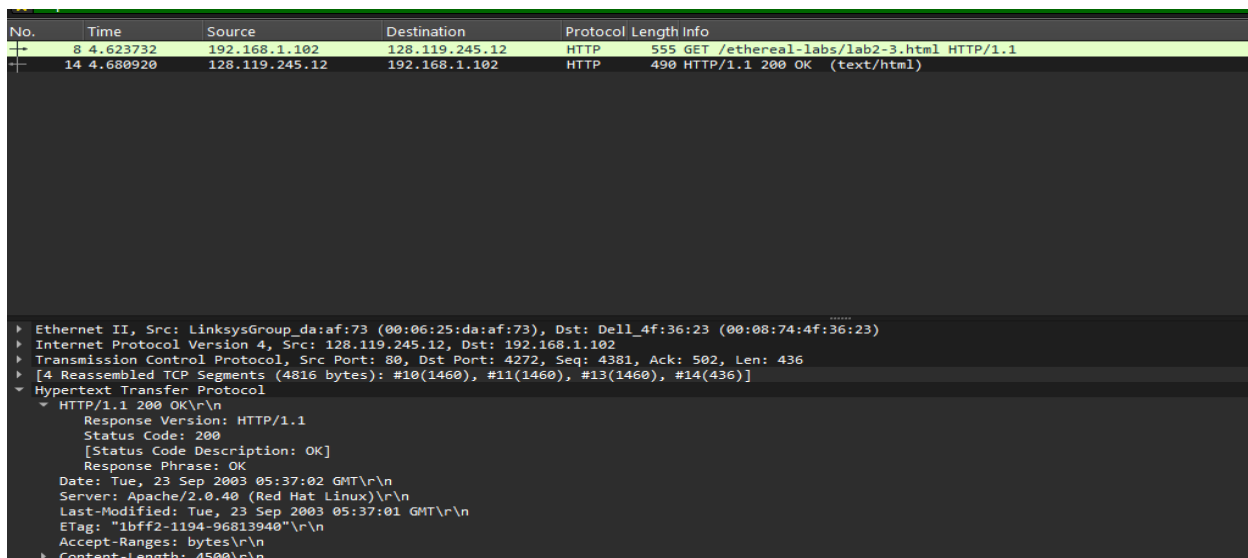
6. Which packet number in the trace contains the GET message for The Bill of Rights?

From the trace details, the **GET message for The Bill of Rights** is contained in **Packet 8**.



7. Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request?

The **status code and phrase in the response** to the HTTP GET request are in **Packet 14**.



8. What is the status code and phrase in the response?

The **status code** in the response is **200**, and the **phrase** is **OK**. This means the request was successful.

No.	Time	Source	Destination	Protocol	Length	Info
8	4.623732	192.168.1.102	128.119.245.12	HTTP	555	GET /ethereal-labs/lab2-3.html HTTP/1.1
14	4.680920	128.119.245.12	192.168.1.102	HTTP	490	HTTP/1.1 200 OK (text/html)


```
.....
▶ Ethernet II, Src: LinksysGroup_da:af:73 (00:06:25:da:af:73), Dst: Dell_4f:36:23 (00:08:74:4f:36:23)
▶ Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.1.102
▶ Transmission Control Protocol, Src Port: 80, Dst Port: 4272, Seq: 4381, Ack: 502, Len: 436
▶ [4 Reassembled TCP Segments (4816 bytes): #10(1460), #11(1460), #13(1460), #14(436)]
▼ Hypertext Transfer Protocol
  ▼ HTTP/1.1 200 OK\r\n
    Response Version: HTTP/1.1
    Status Code: 200
    [Status Code Description: OK]
    Response Phrase: OK
    Date: Tue, 23 Sep 2003 05:37:02 GMT\r\n
    Server: Apache/2.0.40 (Red Hat Linux)\r\n
    Last-Modified: Tue, 23 Sep 2003 05:37:01 GMT\r\n
    ETag: "1bffa2-1194-96813940"\r\n
    Accept-Ranges: bytes\r\n
    Content-Length: 4500\r\n
```

9. How many data-containing TCP segments were needed to carry the single HTTP response and the text of the Bill of Rights? What are the numbers of those packets?

The **single HTTP response and the text of the Bill of Rights** are carried in **4 data-containing TCP segments**. These segments are in **Packets 10, 11, 13, and 14**.

```
TCP payload (436 bytes)
TCP segment data (436 bytes)
[4 Reassembled TCP Segments (4816 bytes): #10(1460), #11(1460), #13(1460), #14(436)]
Hypertext Transfer Protocol
```

In-Lab Statement 3: Trick Question

What is the length of the text for The Bill of Rights in bytes? How do you justify this length of text when your Response Packet Size is only 490 bytes?

- The **length of the text** for The Bill of Rights is **4500 bytes**, as specified in the **Content-Length** header of the HTTP response. However, the response packet in **Packet 14** only contains **490 bytes**.

Explanation:

- The HTTP response is divided across multiple TCP segments. In this case, **Packets 10, 11, 13, and 14** combine to form the complete response, reassembling to a total of **4816 bytes**. Out of these, 4500 bytes represent the actual text of The Bill of Rights, while the remaining bytes account for the HTTP headers.
- The reason the **response packet size is smaller** (490 bytes) is that HTTP data can be broken into smaller segments and transmitted in parts. TCP takes care of assembling these segments upon arrival to deliver the full 4500 bytes of content.